arXiv:2410.02958v2 [cs.LG] 6 Jun 2025

AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML

Patara Trirat [1] **Wonyong Jeong** [1] **Sung Ju Hwang** [1 2]

Abstract

Automated machine learning (AutoML) accelerates AI development by automating tasks in the development pipeline, such as optimal model search and hyperparameter tuning. Existing AutoML systems often require technical expertise to set up complex tools, which is in general time-consuming and requires a large amount of human effort. Therefore, recent works have started exploiting large language models (LLM) to lessen such burden and increase the usability of AutoML frameworks via a natural language interface, allowing non-expert users to build their data-driven solutions. These methods, however, are usually designed only for a particular process in the AI development pipeline and do not efficiently use the inherent capacity of the LLMs. This paper proposes *AutoML-Agent*, a novel multi-agent framework tailored for full-pipeline AutoML, i.e., from data retrieval to model deployment. *AutoML-Agent* takes user's task descriptions, facilitates collaboration between specialized LLM agents, and delivers deployment-ready models. Unlike existing work, instead of devising a single plan, we introduce a retrieval-augmented planning strategy to enhance exploration to search for more optimal plans. We also decompose each plan into subtasks (e.g., data preprocessing and neural network design) each of which is solved by a specialized agent we build via prompting executing in parallel, making the search process more efficient. Moreover, we propose a multi-stage verification to verify executed results and guide the code generation LLM in implementing successful solutions. Extensive experiments on seven downstream tasks using fourteen datasets show that *AutoML-Agent* achieves a higher success rate in automating the full AutoML process, yielding systems with good performance throughout the diverse domains.

[1]DeepAuto.ai [2]KAIST, Seoul, South Korea. Correspondence to: Sung Ju Hwang <sjhwang@deepauto.ai>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

1. Introduction

Automated machine learning (AutoML) has significantly reduced the need for technical expertise and human labors in developing effective data-driven solutions by automating each process in the AI development pipeline (Yao et al., 2018; Ren et al., 2020; He et al., 2021), such as feature engineering, model selection, and hyperparameter optimization (HPO). However, current AutoML systems (Gijsbers et al., 2024) often necessitate programming expertise to configure complex tools and resources, potentially creating barriers for a larger pool of users with limited skills and knowledge, such as domain experts (Sun et al. (2023); §A).

To make AutoML frameworks more accessible to non-expert users, many recent studies (Trirat et al., 2021; Viswanathan et al., 2023; Li et al., 2023; Hollmann et al., 2023b; Liu et al., 2025; Zhang et al., 2023; Shen et al., 2023; Zhang et al., 2024a; Hong et al., 2024a; Guo et al., 2024a; Yang et al., 2025; Chi et al., 2024) have suggested to use natural language interfaces with large language models (LLM) for machine learning (ML) and data science (DS) tasks. Nevertheless, these previous LLM-based AutoML frameworks only considered a limited number of tasks due to their restricted designs, either only for a process in the pipeline (e.g., feature engineering (Hollmann et al., 2023b; Li et al., 2024; Malberg et al., 2024), HPO (Liu et al., 2024a; 2025; Zhang et al., 2024a), and model selection (Zhang et al., 2023; Shen et al., 2023)) or for a specific group of downstream tasks (e.g., natural language processing (Viswanathan et al., 2023) and computer vision (Yang et al., 2025)). In addition, most methods overlook the inherent capability of LLMs to search for promising models by performing actual training of the candidate models during the search process, making it prohibitively costly and slow.

For an AutoML framework to be truly practical, it should perform end-to-end AutoML, considering both the **data aspects** (retrieval, preprocessing, and feature engineering) and **model aspects** (selection, HPO, and deployment). This is because a process in one aspect can affect subsequent processes in the other, potentially leading to suboptimal solutions when combining results from different frameworks. Meanwhile, the AutoML framework should be computationally efficient, using strategies to minimize the computational overhead during search. However, there are two main challenges in building such a framework.